

## Introduction to Java



Java is a **high-level, class-based, object-oriented general-purpose programming language** created by **James Gosling** and his team at **Sun Microsystems** in **1995**.

At its core, Java was designed to be **secure, robust, portable and platform – independent**.

Java follows the principle of **WORA (Write Once, Run Anywhere)**, where **source code is compiled into bytecode** that can run on any system with a **Java Virtual Machine (JVM)**

## Features of Java



- ⇒ High-Level Language
- ⇒ Object-Oriented Programming (OOP)
- ⇒ Statically Typed
- ⇒ Compiled and interpreted
- ⇒ Platform independent
- ⇒ Achieves WORA (Write Once, Run Anywhere)
- ⇒ Automatic Memory Management using the Garbage Collector (GC)

## Applications of Java



- ⇒ Backend development
- ⇒ Desktop application development
- ⇒ Web application development
- ⇒ Data-driven applications
- ⇒ Enterprise applications
- ⇒ Android application development

## **System.out.println();**



- **System** is a class in the **java.lang** package.
- **java.lang** package is automatically imported by the Java compiler.
- **out** is a **public static final PrintStream object** inside the **System class**.
- **println()** is a method of the **PrintStream** class that prints the data and moves the cursor to the next line.

## **System.out.print();**



- **System** is a class in the **java.lang** package.
- **java.lang** package is automatically imported by the Java compiler.
- **out** is a **public static final PrintStream object** inside the **System class**.
- **print()** is a method of the **PrintStream class** that prints the given data and keeps the cursor on the **same line** after printing.

## **Tokens in Java**

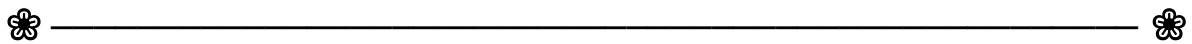


A **token** is the **smallest individual unit** of a Java program that has **meaningful information** to the Java compiler.

**The main types of tokens are:**

1. Reserved Words (Keywords)
2. Identifiers
3. Operators
4. Punctuators (Separators)
5. Literals

## ↳ Keywords:



**keywords** are reserved words that have predefined meaning and they are used to perform specific tasks.

**Some examples of Java keywords are:**

1. **if**
2. **while**
3. **class**
4. **public**
5. **static**
6. **void**
7. **return**

**Note:** **true**, **false**, and **null** are reserved literals, not keywords.

## ↳ Identifiers:

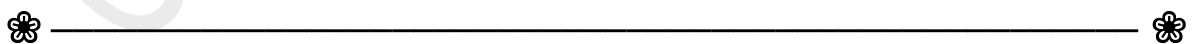


An identifier is a **user-defined name** given to variables, methods, classes, interfaces, packages, or other program elements to identify them uniquely.

**Naming Rules for Identifiers in Java:**

1. An identifier cannot start with a digit (0–9).
2. An identifier cannot contain spaces.
3. An identifier cannot contain special characters except the underscore ( `_` ) and dollar sign ( `$` ).
4. Java identifiers are case-sensitive.
5. An identifier cannot be a Java keyword (**reserved word**).

## ↳ Literals in Java:

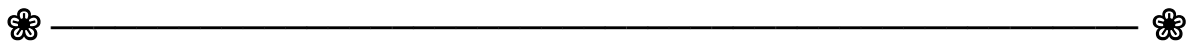


Literals are **fixed values** that are directly assigned to variables or used in a Java program.

**Examples of literals include:**

- ✓ Numeric literals (e.g., 10, 3.14)
- ✓ Character literals (e.g., 'A')
- ✓ String literals (e.g., "Hello")
- ✓ Boolean literals (true, false)
- ✓ Null literal (null)

## └─► Punctuators / (Separators):



**Punctuators are special symbols** in Java program used to **structure and organize the code**.

### Common Punctuators in Java

- ; Semicolon (Statement Terminator)
- , Comma (Separator)
- . Dot (Member Access/Package Separator)
- () Parentheses (Method Calls, Parameters, Conditions)
- {} Curly Braces (Block of Code)
- [] Square Brackets (Arrays)
- : Colon (Used in Ternary Operator, Enhanced for-Loop, Switch Labels)

## Public class in Java

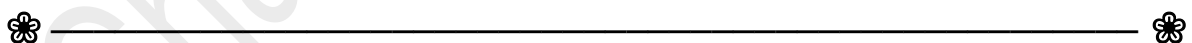


**public** is an **access modifier** that makes a **class accessible from anywhere in the program**.

**Class names should follow the PascalCase** naming convention, where the **first letter of each word is capitalized**.

**Example:** - StudentLibraryManagement

## └─► Rules of a public Class in Java:



- ✓ The **file name must exactly match the public class name**.
- ✓ A Java source file can contain **only one public class**.
- ✓ The public keyword must be written **before** the class keyword.
- ✓ The class name must be a **valid Java identifier**.

**Example:**

```
J Demo.java X README.md
src > com > channa > JavaPractice > J Demo.java > Demo
1 package com.channa.JavaPractice; //package
2
3 public class Demo {
4
5 }
```

## Main methods in Java



The **main()** method is the entry point of a Java application the JVM starts executing the program by **invoking the main()** method.

### ► Breakdown of the main() Method

**1. public:** is an access modifier that allows the **JVM** to access the **main()** method from anywhere.

**2. static:** means the **main()** method belongs to the class, so the **JVM** can call it **without creating an object** of the class.

**3. void:** is the **return type**, which indicates that the **main()** method does not return any value.

**4. main:** is the **predefined method name** that the **JVM** looks for as the starting point of a Java application.

**5. String[] args:** is an **array of strings** used to receive **command-line arguments** passed to the program.

**Syntax:**

```
public static void main(String[] args) {  
    //statements  
}
```

### Example:

The screenshot shows an IDE with a file named 'Demo.java' open. The code is as follows:

```
1 package com.channa.JavaPractice; //package  
2  
3 public class Demo {  
4     public static void main(String[] args) {  
5         System.out.println("Hello, World!");  
6     }  
7 }
```

Below the code editor, there is a 'Terminal' window showing the command prompt output:

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program  
Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' '-XX:  
+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDr  
ive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'  
Hello, World!  
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

# Variables in Java



A variable is a named **memory container** used to **store data** that can be **accessed and manipulated** during program execution.

In Java, variable names are **case-sensitive**.

Variable names should be valid **Java identifiers**.

```
J Demo.java 1 X  README.md
src > com > channa > JavaPractice > J Demo.java > Demo
1 package com.channa.JavaPractice; //package
2
3 public class Demo {
4     Run | Debug
     public static void main(String[] args) {
5         int age = 10; // data type int, age is the actual variable or memory location, 22 is integer type of data.
6     }
7 }
```

## ➤ Variable Declaration:



Variable declaration is the process of creating a variable by **specifying its data type and name**.

**Example:**

```
public class Demo {
    Run | Debug
    public static void main(String[] args) {
        int age;
    }
}
```

## ➤ Variable Initialization:



Variable initialization is the process of assigning a value to a variable.

**Example:**

```
J Demo.java X  README.md
src > com > channa > JavaPractice > J Demo.java > Demo > main(String[])
1 package com.channa.JavaPractice; //package
2
3 public class Demo {
4     Run | Debug
     public static void main(String[] args) {
5         int age;
6         age = 22;
7         System.out.println("Age is: " + age);
8     }
9 }
```

Chat

Terminal

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'c:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' -XX:ShowCodeDetailsInExceptionMessages -cp 'c:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
Age is: 22
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

**Note: Java allows a variable to be declared and initialized in the same statement.**

## Many Values to Multiple Variables



Java allows multiple values to multiple variables within the same statement, but each variable must be explicitly assigned its value.

**Syntax:** datatype variable1 = value1, variable2 = value2, variable3 = value3;

**Example:**

```
1 package com.channa.JavaPractice;
2
3 public class Student {
4     public static void main(String[] args){
5         int num1 = 10, num2 = 20, num3 = 30;
6         System.out.println( num1);
7         System.out.println( num2);
8         System.out.println( num3);
9     }
10 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\re-dhat.java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Student'
10
20
30
PS C:\Users\chann\OneDrive\Desktop\Java>
```

## Single Value to Multiple Variables



Java allows a single value to be assigned to multiple variables in a single statement using chained assignment.

**Syntax:** datatype variable1 = value1, variable2 = value2;

**Example:**

```
1 package com.channa.JavaPractice;
2
3 public class Student {
4     public static void main(String[] args){
5
6         // int num1 = num2 = num3 = 10; CTE
7         //Java does not allow declaration and chained assignment together
8
9         //Java allows chained assignment after the variables are declared.
10        int num1, num2, num3;// declaration
11        num1 = num2 = num3 = 10; // assignment
12        System.out.println( num1);
13        System.out.println( num2);
14        System.out.println( num3);
15    }
16 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\re-dhat.java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Student'
10
10
10
PS C:\Users\chann\OneDrive\Desktop\Java>
```

# Operators in Java



Operators are **predefined symbols** used to perform operations on operands (values or variables)

**In Java, operators are classified into seven main types:**

1. Arithmetic Operators
2. Assignment Operators
3. Relational (Comparison) Operators
4. Logical Operators
5. Ternary (Conditional) Operator
6. Unary Operators
7. Bitwise Operators

## ➤ Arithmetic Operators:



Arithmetic operators are predefined symbols used to perform mathematical operations on operands (values or variables).

**Arithmetic Operators are classified into 5 types in java:**

- ✓ Addition (+)
- ✓ Subtraction (-)
- ✓ Multiplication (\*)
- ✓ Division (/) and
- ✓ Modulus (%).

**Example program:**

```
src > com > channa > JavaPractice > Demo.java > {} com.channa.JavaPractice
1 package com.channa.JavaPractice;
2
3 public class Demo {
4     public static void main(String[] args){
5         int num1 = 5;
6         int num2 = 10;
7         System.out.println("Addition: " + (num1 + num2));
8         System.out.println("Subtraction : " + (num1 - num2));
9         System.out.println("Multiplication : " + (num1 * num2));
10        System.out.println("Division: " + (num1 / num2));
11        System.out.println("Modulus: " + (num1 % num2));
12    }
13 }
14
```

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> cd src
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\src> javac com\chann
a\JavaPractice\Demo.java
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\src> java com.channa
.JavaPractice.Demo
Addition: 15
Subtraction :-5
Multiplication :50
Division: 0
Modulus: 5
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\src>
```

## ➤ Assignment Operators in Java:



Assignment operators are used to assign values or the result of an expression to variables.

**In Java, the basic assignment operators are:**

- ✓ = Simple Assignment
- ✓ += Add and Assign
- ✓ -= Subtract and Assign
- ✓ \*= Multiply and Assign
- ✓ /= Divide and Assign
- ✓ %= Modulus and Assign

**Example program:**

```
Demo.java X README.md
src > com > channa > JavaPractice > Demo.java Demo main(String[])
1 package com.channa.JavaPractice;
2
3 public class Demo {
4     public static void main(String[] args){
5         int laddus = 5;
6         laddus += 25;
7         laddus -= 20;
8         laddus *= 2;
9         System.out.println("Total laddus are: " + laddus);
10    }
11 }
```

```
Chat
Terminal
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> cd C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\src
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\src> javac com\channa\JavaPractice\Demo.java
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\src> java com.channa.JavaPractice.Demo
Total laddus are: 20
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\src>
```

## ➤ Relational Operators:



Relational operators are used to compare two operands and return a Boolean result like (true or false).

**Types of Relational Operators:**

- > Greater Than
- < Less Than
- >= Greater Than or Equal To
- <= Less Than or Equal To
- == Equal to
- != Not Equal to

**Example program:**

```
Demo.java X README.md
src > com > channa > JavaPractice > Demo.java Demo main(String[])
1 package com.channa.JavaPractice;
2
3 public class Demo {
4     public static void main(String[] args){
5         int num1 = 10;
6         int num2 = 20;
7         System.out.println("> := " + (num1 > num2));
8         System.out.println("< := " + (num1 < num2));
9         System.out.println(">= := " + (num1 >= num2));
10        System.out.println("<= := " + (num1 <= num2));
11        System.out.println("== := " + (num1 == num2));
12        System.out.println("!= := " + (num1 != num2));
13    }
14 }
```

```
Chat
Terminal
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
> := false
< := true
>= := false
<= := true
== := false
!= := true
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

## ➤ Unary Operators in Java:



Unary operators are used to operate on a single operand to perform a specific operation.

### The main Types of Unary Operators are used in Java are:

- ++ Increment
- Decrement

➤ **Unary Increment Operator:** is used to increase the value by one.

**Increment Operator** have two types:

1. Pre-Increment Operator
2. Post-Increment Operator

➤ **Pre-Increment Operator:** means first increase the value by one then use it.

### Example:

```
src > com > channa > JavaPractice > Demo.java > Demo > main(String[])
1 package com.channa.JavaPractice;
2
3 public class Demo {
4     public static void main(String[] args){
5         int num = 10;
6         int temp = ++num; // 10 increased by 1 then assigned to temp.
7         System.out.println(temp);
8         System.out.print(num);
9     }
10 }
11
```

Terminal

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adopti
um\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
11
11
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

➤ **Post-Increment Operator:** means first use the value then Increase by one.

### Example:

```
src > com > channa > JavaPractice > Demo.java > Demo > main(String[])
1 package com.channa.JavaPractice;
2
3 public class Demo {
4     public static void main(String[] args){
5         int num = 10;
6         int temp = num++; // 10 assigned to temp then increased by 1
7         System.out.println(temp);
8         System.out.print(num);
9     }
10 }
11
```

Terminal

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adopti
um\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
10
11
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

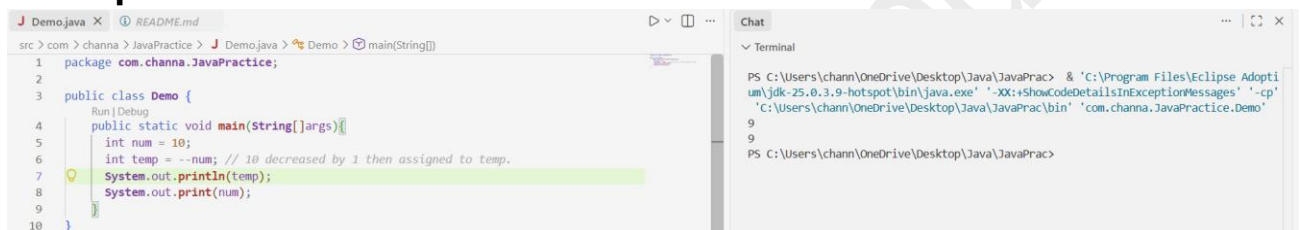
☞ **Unary Decrement Operator:** is used to decrease the value by one.

**Decrement Operator have two types:**

1. Pre-Decrement Operator
2. Post-Decrement Operator

☞ **Pre-Decrement Operator:** means first decrease the value by one then use it.

**Example:**



The screenshot shows an IDE with a Java file named Demo.java. The code is as follows:

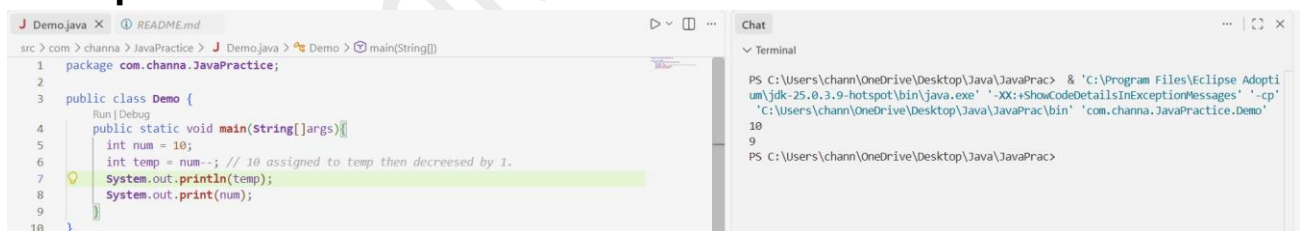
```
1 package com.channa.JavaPractice;
2
3 public class Demo {
4     public static void main(String[] args) {
5         int num = 10;
6         int temp = --num; // 10 decreased by 1 then assigned to temp.
7         System.out.println(temp);
8         System.out.print(num);
9     }
10 }
```

The IDE also shows a terminal window with the following output:

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adopti
um\jdk-25.0.3-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
9
9
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

☞ **Post-Decrement Operator:** means first use the value then decrease by one.

**Example:**



The screenshot shows an IDE with a Java file named Demo.java. The code is as follows:

```
1 package com.channa.JavaPractice;
2
3 public class Demo {
4     public static void main(String[] args) {
5         int num = 10;
6         int temp = num--; // 10 assigned to temp then decreased by 1.
7         System.out.println(temp);
8         System.out.print(num);
9     }
10 }
```

The IDE also shows a terminal window with the following output:

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adopti
um\jdk-25.0.3-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
10
9
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

## ➤ Logical Operators in Java:



Logical operators are used to combine or negate conditional expressions. The result of a logical operation is **true** or **false**.

### Operators:

**&& (and)** = true if both the conditions are true.  
**|| (or)** = true if at least one condition is true.  
**! (not)** = It reverses the result.

➤ **&& (Logical AND):** The logical AND operator returns true only if both the given conditions are true; otherwise, it returns false.

### Truth table:

| A     | B     | C            |
|-------|-------|--------------|
| True  | True  | <b>True</b>  |
| True  | False | <b>False</b> |
| False | True  | <b>False</b> |
| False | False | <b>False</b> |

### Example:

```
src > com > channa > JavaPractice > Demo.java > Demo > main(String[])
1 package com.channa.JavaPractice;
2
3 public class Demo {
4     public static void main(String[] args) {
5         int num1 = 10;
6         int num2 = 11; // 10 assigned to temp then decreased by 1.
7         boolean result = num1++ > num2++ || num1++ < ++num2;
8         System.out.println(result);
9     }
10 }
```

Terminal Output:

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adopti
um\jdk-25.0.3.9-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages '-cp'
'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
true
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

➤ **|| (Logical OR):** The logical OR operator returns true if at least one of the given conditions is true.

### Truth table:

| A     | B     | C            |
|-------|-------|--------------|
| True  | True  | <b>True</b>  |
| True  | False | <b>True</b>  |
| False | True  | <b>True</b>  |
| False | False | <b>False</b> |

## Example:

```
src > com > channa > JavaPractice > J Demo.java > Demo > main(String[] args)
1 package com.channa.JavaPractice;
2
3 public class Demo {
4     public static void main(String[] args){
5         int num1 = 10;
6         int num2 = 11; // 10 assigned to temp then decreased by 1.
7         boolean result = num1++ > num2++ && num1++ < ++num2;
8         System.out.println(result);
9     }
10 }
```

Chat

Terminal

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
false
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

! (Logical NOT): The logical NOT, is also known as negation operator, used to invert or negate a boolean value.

## Truth table:

| Expression | Result |
|------------|--------|
| True       | False  |
| False      | True   |

## Example:

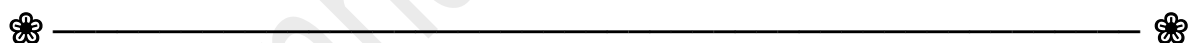
```
src > com > channa > JavaPractice > J Demo.java > Demo > main(String[] args)
1 package com.channa.JavaPractice;
2
3 public class Demo {
4     public static void main(String[] args){
5         int num1 = 10;
6         int num2 = 10; // 10 assigned to temp then decreased by 1.
7         boolean result = num1 == num2;
8         System.out.println(!result);
9     }
10 }
```

Chat

Terminal

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
false
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

## Ternary (Conditional) Operator in Java:



The **ternary operator** is a **conditional operator that evaluates** a condition and **returns one of two expressions** depending on whether the condition is **true or false**.

**Syntax:** (condition) ? expression1 : expression2;

## Example:

```
src > com > channa > JavaPractice > J Demo.java > Demo
1 package com.channa.JavaPractice;
2
3 public class Demo {
4     public static void main(String[] args){
5         int age = 18;
6         String result = (age >= 18)? "Channa is eligibal for vote" : "Channa is not eligibal for vote";
7         System.out.print("Result is: " + result);
8     }
9 }
```

Chat

Terminal

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
Result is: Channa is eligibal for vote
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

## Operator precedence in Java



**Operator precedence** determines **which operator is evaluated first** when an expression contains multiple operators.

**Example:** `int result = 10 + 5 * 2;`

**Here, \* has higher precedence than +, so multiplication happens first:**  
→ Higher precedence → evaluated first.

**Note:** if two operators have the **same precedence**, Java uses **associativity** to determine the evaluation direction.

### ↳ **Associativity in Operator precedence:**



**Associativity** determines the **direction in which operators with the same precedence are evaluated**.

Java has **two associativity directions**:

- ↳ **Left-to-right associativity**
- ↳ **Right-to-left associativity**

### ↳ **Left-to-right associativity**

**+** and **-** have the **same precedence**, and their associativity is **left-to-right**.

**Example:** `int result = 20 - 5 + 2;`

Java evaluates:  $(20 - 5) + 2$   
 $15 + 2 = 17$

**Note:** **+**, **-**, **\***, and **/** use **left-to-right associativity**.

### ➤ Right-to-left associativity

**Example:** `int a, b, c;  
a = b = c = 10;`

Java groups it **from right to left**: `a = (b = (c = 10))`

So:

1. `c = 10`
2. `b = 10`
3. `a = 10`

Therefore, **= assignment operator** has right-to-left associativity.

### Java Operator Precedence — Highest – To – Lowest

| Priority | Operators                                              | Category              | Associativity |
|----------|--------------------------------------------------------|-----------------------|---------------|
| 1        | <code>() [] .</code>                                   | Postfix / access      | Left to Right |
| 2        | <code>expr++, expr--</code>                            | Postfix               | Left to Right |
| 3        | <code>++expr, --expr +, -, !, ~</code>                 | Unary                 | Right to Left |
| 4        | <code>*, /, %</code>                                   | Multiplicative        | Left to Right |
| 5        | <code>+, -</code>                                      | Additive              | Left to Right |
| 6        | <code>&lt;&lt;, &gt;&gt;, &gt;&gt;&gt;</code>          | Shift                 | Left to Right |
| 7        | <code>&lt;, &gt;, &lt;=, &gt;= &amp; instanceof</code> | Relational            | Left to Right |
| 8        | <code>==, !=</code>                                    | Equality              | Left to Right |
| 9        | <code>&amp; ^  </code>                                 | Bitwise AND, XOR, OR  | Left to Right |
| 10       | <code>&amp;&amp;,   </code>                            | Logical AND, OR       | Left to Right |
| 11       | <code>?:</code>                                        | Ternary / Conditional | Right to Left |
| 12       | <code>=, +=, -=, *=, /=, %= etc.</code>                | Assignment            | Right to Left |
| 13       | <code>-&gt;</code>                                     | Lambda                | Right to Left |

**Note:** In Java, parentheses () have the highest precedence. By grouping operators using parentheses, we can change the direction/order of evaluation.

### Example:

```
1 package com.channa.JavaPractice;
2
3 public class Student {
4     public static void main(String[] args){
5         int num1 = 10;
6         int num2 = 5;
7         // without grouping expression will execute from left to right.
8         int result = num1 - (num2 - 2);
9         System.out.println("result is: " + result);
10    }
11 }
```

PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt\_ws\Java\_e2daa7ee\bin' 'com.channa.JavaPractice.Student'  
result is: 7  
PS C:\Users\chann\OneDrive\Desktop\Java>

## Expression in Java

Expression in Java is a combination of variables, values, and operators that evaluates and produces a single value.

**Example:** `int result = 10 + 5 * 2;`

**Here:**

- 10, 5, 2 are values
- +, \* are operators
- 10 + 5 \* 2 is an expression

☞ The expression evaluates to 20 (**single value**)

## Data types in Java

A data type specifies the type of data a variable can store, the amount of memory allocated, the range of values it can hold and what operations can be performed on that value.

**In Java, data types are classified into two types:**

1. Primitive Data Types.
2. Non-Primitive (Reference) Data Types: **Classes, objects, interfaces, strings etc.**

## ↳► Primitive data types in Java:



**Primitive data types** are **built-in data types** provided by Java that store single value.

In Java, **primitive data types** are classified into **8 types**:

| Primitive Data Types | Size              | Default value |
|----------------------|-------------------|---------------|
| <b>byte</b>          | 1 byte (8 bits)   | 0             |
| <b>short</b>         | 2 bytes (16 bits) | 0             |
| <b>char</b>          | 2 bytes (16 bits) | '\u0000'      |
| <b>int</b>           | 4 bytes (32 bits) | 0             |
| <b>long</b>          | 8 bytes (64 bits) | 0L            |
| <b>float</b>         | 4 bytes (32 bits) | 0.0f          |
| <b>double</b>        | 8 bytes (64 bits) | 0.0d          |
| <b>boolean</b>       | JVM-dependent     | false         |

## ↳► Bits and Bytes in Java:



↳► **Bit:** A **bit (binary digit)** is the smallest unit of data in a computer that can store either **0** or **1**.

↳► **Byte:** A **byte** is a unit of memory consisting of **8 bits**.

### Quick Memory Trick

- 1 Bit = 0 or 1
- 8 Bits = 1 Byte
- 1024 Bytes = 1 KB
- 1024 KB = 1 MB
- 1024 MB = 1 GB
- 1024 GB = 1 TB

## ➤ ASCII Values:



### ASCII (American Standard Code for Information Interchange)

It is a character encoding standard that assigns a unique numeric value to each character.

### Why do we need ASCII?

Computers understand only **binary (0s and 1s)**, not characters like **A, B**, or **@**.

ASCII provides a numeric code for each character so the computer can store and process it.

### ASCII values are:

| Decimal | Character | Description          |
|---------|-----------|----------------------|
| 0       | NUL       | Null                 |
| 1       | SOH       | Start of Heading     |
| 2       | STX       | Start of Text        |
| 3       | ETX       | End of Text          |
| 4       | EOT       | End of Transmission  |
| 5       | ENQ       | Enquiry              |
| 6       | ACK       | Acknowledge          |
| 7       | BEL       | Bell                 |
| 8       | BS        | Backspace            |
| 9       | TAB       | Horizontal Tab       |
| 10      | LF        | Line Feed (New Line) |
| 11      | VT        | Vertical Tab         |
| 12      | FF        | Form Feed            |
| 13      | CR        | Carriage Return      |
| 14      | SO        | Shift Out            |
| 15      | SI        | Shift In             |
| 16      | DLE       | Data Link Escape     |
| 17      | DC1       | Device Control 1     |
| 18      | DC2       | Device Control 2     |

| Decimal | Character | Description               |
|---------|-----------|---------------------------|
| 19      | DC3       | Device Control 3          |
| 20      | DC4       | Device Control 4          |
| 21      | NAK       | Negative Acknowledge      |
| 22      | SYN       | Synchronous Idle          |
| 23      | ETB       | End of Transmission Block |
| 24      | CAN       | Cancel                    |
| 25      | EM        | End of Medium             |
| 26      | SUB       | Substitute                |
| 27      | ESC       | Escape                    |
| 28      | FS        | File Separator            |
| 29      | GS        | Group Separator           |
| 30      | RS        | Record Separator          |
| 31      | US        | Unit Separator            |
| 32      | Space     | Space                     |
| 33      | !         | Exclamation Mark          |
| 34      | "         | Double Quote              |
| 35      | #         | Hash                      |
| 36      | \$        | Dollar                    |
| 37      | %         | Percent                   |
| 38      | &         | Ampersand                 |
| 39      | '         | Apostrophe                |
| 40      | (         | Left Parenthesis          |
| 41      | )         | Right Parenthesis         |
| 42      | *         | Asterisk                  |
| 43      | +         | Plus                      |
| 44      | ,         | Comma                     |
| 45      | -         | Hyphen                    |
| 46      | .         | Period                    |
| 47      | /         | Slash                     |
| 48      | 0         | Digit Zero                |
| 49      | 1         | Digit One                 |
| 50      | 2         | Digit Two                 |

| Decimal | Character | Description          |
|---------|-----------|----------------------|
| 51      | 3         | Digit Three          |
| 52      | 4         | Digit Four           |
| 53      | 5         | Digit Five           |
| 54      | 6         | Digit Six            |
| 55      | 7         | Digit Seven          |
| 56      | 8         | Digit Eight          |
| 57      | 9         | Digit Nine           |
| 58      | :         | Colon                |
| 59      | ;         | Semicolon            |
| 60      | <         | Less Than            |
| 61      | =         | Equal                |
| 62      | >         | Greater Than         |
| 63      | ?         | Question Mark        |
| 64      | @         | At Symbol            |
| 65–90   | A–Z       | Uppercase Letters    |
| 91      | [         | Left Square Bracket  |
| 92      | \         | Backslash            |
| 93      | ]         | Right Square Bracket |
| 94      | ^         | Caret                |
| 95      | _         | Underscore           |
| 96      | `         | Backtick             |
| 97–122  | a–z       | Lowercase Letters    |
| 123     | {         | Left Curly Brace     |
| 124     |           | Vertical Bar (Pipe)  |
| 125     | }         | Right Curly Brace    |
| 126     | ~         | Tilde                |
| 127     | DEL       | Delete               |

**For java IMP are 0 – 9, A – Z & a – z**

# Type Casting



Type casting is the process of converting a value or variable from one data type to another.

## Java supports two types of type casting:

- Widening (Implicit Type Casting)
- Narrowing (Explicit Type Casting)

### ➤ Widening Type Casting (Implicit):



**Widening** is the **automatic conversion** of a **smaller data type** to a **larger compatible data type** without data loss.

#### Example:

```
src > com > channa > JavaPractice > Demo.java > Demo
1 package com.channa.JavaPractice; //package
2
3 public class Demo {
4     public static void main(String[] args) {
5         int num1 = 10;
6         double num2 = num1;
7         System.out.print("Result: " + num2);
8     }
9 }
```

Terminal Output: PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo' Result: 10.0

### ➤ Narrowing Type Casting (Explicit):



**Narrowing type casting** is the **manual conversion** of a **larger data type** to a **smaller compatible data type** using the **cast operator ( )**.

#### Example:

```
src > com > channa > JavaPractice > Demo.java > Demo > main(String[])
1 package com.channa.JavaPractice; //package
2
3 public class Demo {
4     public static void main(String[] args) {
5         double num1 = 10.99;
6         int num2 = (int) num1;
7         System.out.print("Result: " + num2);
8     }
9 }
```

Terminal Output: PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo' Result: 10

**Note:** Converting short to char and char to short are both narrowing conversions (explicit type casting).

## Type Promotion in Java



Type promotion occurs when Java automatically promotes the smaller integer types, such as **byte**, **short**, **char**, to **int** during arithmetic operations.

### Example:

```
src > com > channa > JavaPractice > J Demo.java > Demo > main(String[])
1 package com.channa.JavaPractice; //package
2
3 public class Demo {
4     public static void main(String[] args) {
5         byte num1 = 5;
6         short num2 = 6;
7         char num3 = 5;
8         int num4 = num1 + num2 + num3;
9         System.out.println("Result: " + num4);
10    }
11 }
```

Terminal

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
Result: 16
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

## Control Statements in Java



**Control statements** are used to **control the flow of program execution** by making decisions, repeating code, or transferring control.

**In Java, control statements are classified into three types:**

1. **Selection (Decision-Making) Statements**
2. **Iteration (Looping) Statements**
3. **Transfer Control (Jump) Statements**

### ➤ Selection Statements:



**Selection statements** are used to **select and execute a block of code** based on a specified condition.

**In Java, selection statements are classified into five types:**

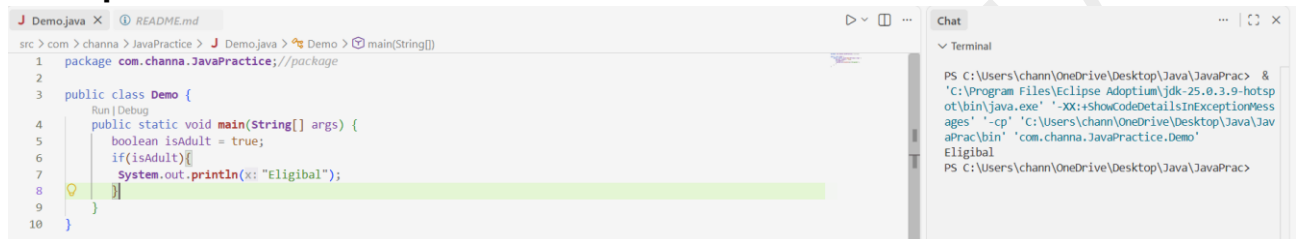
1. **if**
2. **if-else**
3. **else-if ladder**
4. **nested if**
5. **switch**

➤ **if Statement:** is a selection statement used to **select and execute the if block** if the specified condition evaluates to **true**.

**Syntax: -**

```
if (condition) {  
    // statements  
}
```

**Example:**



The screenshot shows an IDE with a Java file named Demo.java. The code is as follows:

```
1 package com.channa.JavaPractice; //package  
2  
3 public class Demo {  
4     public static void main(String[] args) {  
5         boolean isAdult = true;  
6         if(isAdult){  
7             System.out.println("Eligibal");  
8         }  
9     }  
10 }
```

The IDE also shows a terminal window with the following command and output:

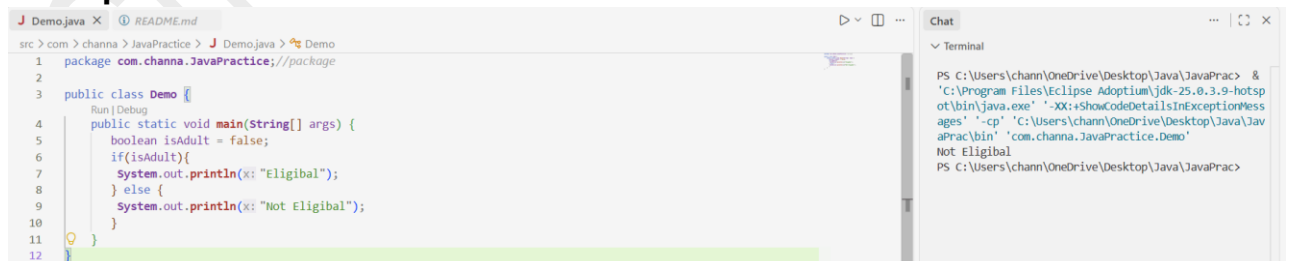
```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> &  
'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotsp  
ot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMess  
ages' -cp 'C:\Users\chann\OneDrive\Desktop\Java\Jav  
aPrac\bin' 'com.channa.JavaPractice.Demo'  
Eligibal  
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

➤ **if – else Statement:** is a selection statement, it executes the **if block**, if the specified condition evaluates to **true** and **else block**, if the specified condition evaluates to **false**.

**Syntax:**

```
if (condition) {  
    // statements  
}  
else {  
    // statements  
}
```

**Example:**



The screenshot shows an IDE with a Java file named Demo.java. The code is as follows:

```
1 package com.channa.JavaPractice; //package  
2  
3 public class Demo {  
4     public static void main(String[] args) {  
5         boolean isAdult = false;  
6         if(isAdult){  
7             System.out.println("Eligibal");  
8         } else {  
9             System.out.println("Not Eligibal");  
10        }  
11    }  
12 }
```

The IDE also shows a terminal window with the following command and output:

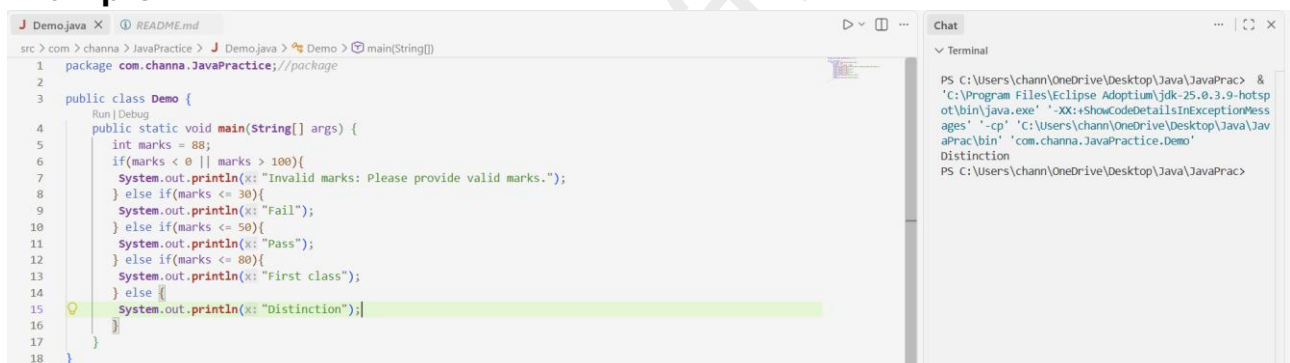
```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> &  
'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotsp  
ot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMess  
ages' -cp 'C:\Users\chann\OneDrive\Desktop\Java\Jav  
aPrac\bin' 'com.channa.JavaPractice.Demo'  
Not Eligibal  
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

➤ **else-if ladder:** is a selection statement used to **check multiple conditions** and **execute the corresponding block** of code based on the **first condition** that evaluates to **true**.

**Syntax:**

```
if (condition1) {  
    // statements  
}  
else if (condition2) {  
    // statements  
}  
else if (condition3) {  
    // statements  
}  
else {  
    // statements  
}
```

**Example:**



The screenshot shows an IDE with a Java file named `Demo.java`. The code defines a `main` method that takes an array of strings as input. It initializes a variable `marks` to 88. Then, it uses an `if-else-if` ladder to check the value of `marks` and print the corresponding grade. The conditions are: `marks < 0 || marks > 100` (Invalid marks), `marks <= 30` (Fail), `marks <= 50` (Pass), `marks <= 80` (First class), and `marks <= 100` (Distinction). The output in the terminal shows the grade "Distinction".

```
1 package com.channa.javaPractice; //package  
2  
3 public class Demo {  
4     public static void main(String[] args) {  
5         int marks = 88;  
6         if(marks < 0 || marks > 100){  
7             System.out.println("Invalid marks: Please provide valid marks.");  
8         } else if(marks <= 30){  
9             System.out.println("Fail");  
10        } else if(marks <= 50){  
11            System.out.println("Pass");  
12        } else if(marks <= 80){  
13            System.out.println("First class");  
14        } else {  
15            System.out.println("Distinction");  
16        }  
17    }  
18 }
```

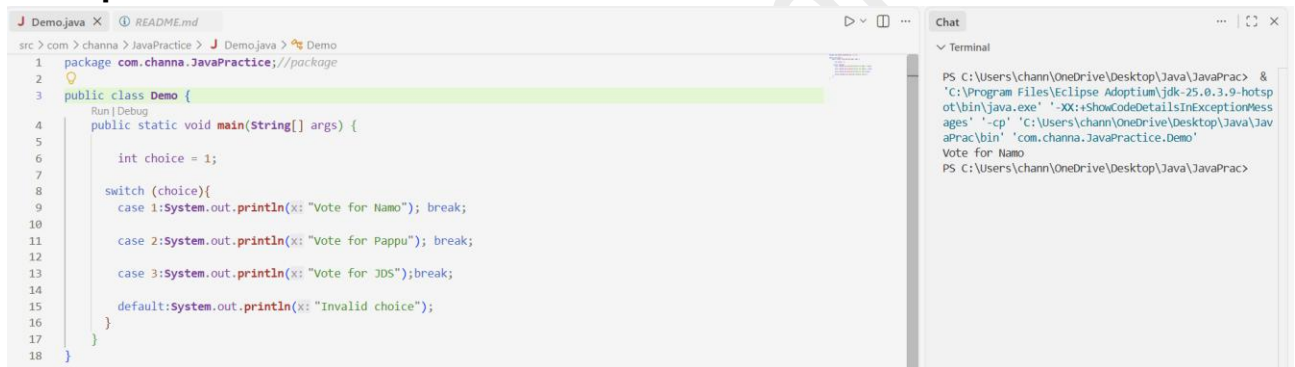
**Note:** Nested if statement means placing an if statement inside another if statement to check a condition inside another.

↳ ➤ **Switch Statement:** is a selection statement used to select and execute one block of code from multiple alternatives based on the value of a specified expression.

**Syntax:**

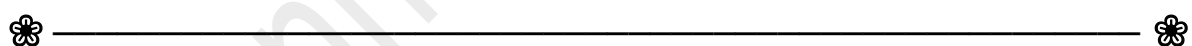
```
switch (expression) {  
    case value1: // statements break;  
  
    case value2: // statements break;  
  
    case value3: // statements break;  
  
    default: // statements  
}
```

**Example:**



The screenshot shows an IDE with a Java file named 'Demo.java'. The code defines a 'Demo' class with a 'main' method. Inside 'main', a variable 'choice' is set to 1. A 'switch' statement follows, with three cases: 'case 1' prints 'Vote for Namu', 'case 2' prints 'Vote for Pappu', and 'case 3' prints 'Vote for JDS'. A 'default' case prints 'Invalid choice'. The terminal on the right shows the command to run the program, followed by the output 'Vote for Namu'.

↳ ➤ **Iteration Statements:**



**Iteration statements** are used to repeatedly **execute a block of code** as long as a specified condition evaluates to **true**.

**In Java, iteration statements are classified into four types:**

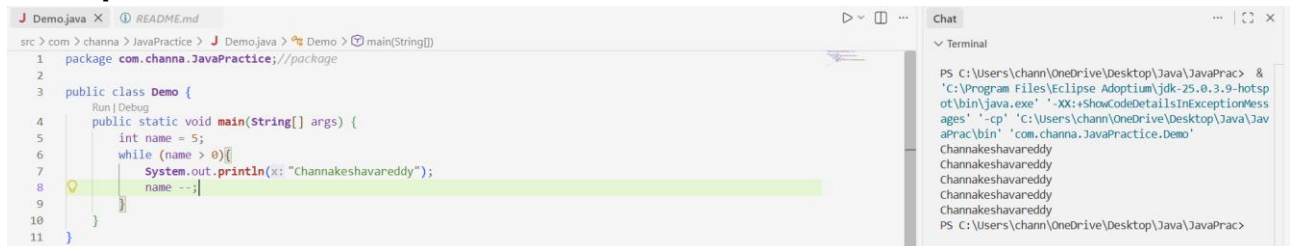
1. while loop
2. do-while loop
3. for loop
4. enhanced for (for-each) loop (We learn in Arrays)

➤ **while loop:** Used to **repeatedly execute a block of code** as long as the specified condition evaluates to **true**, primarily used when the **number of iterations is unknown**.

**Syntax:**

```
while(Condition) {  
    //statements  
}
```

**Example:**



```
1 package com.channa.JavaPractice; //package  
2  
3 public class Demo {  
4     public static void main(String[] args) {  
5         int name = 5;  
6         while (name > 0) {  
7             System.out.println("Channakeshavareddy");  
8             name --;  
9         }  
10    }  
11 }
```

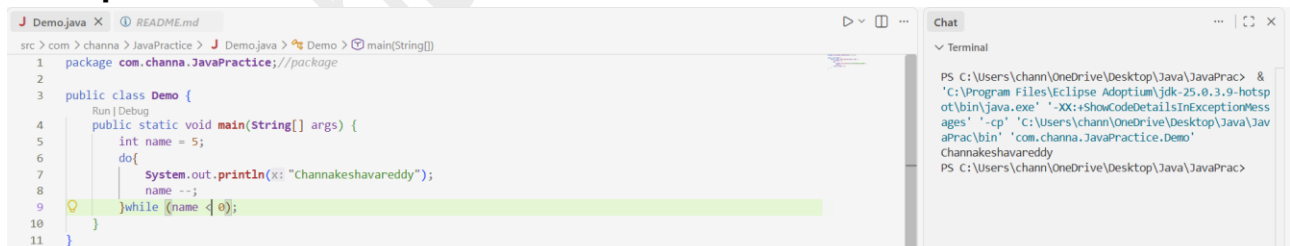
➤ **do-while loop:** is an **exit-controlled iteration statement**, used to execute the loop body at least once, even if the specified condition evaluates to **false**.

**Syntax:**

```
do {  
    // statements  
} while (condition);
```

**Note:** The do-while loop ends with a semicolon (;).

**Example:**



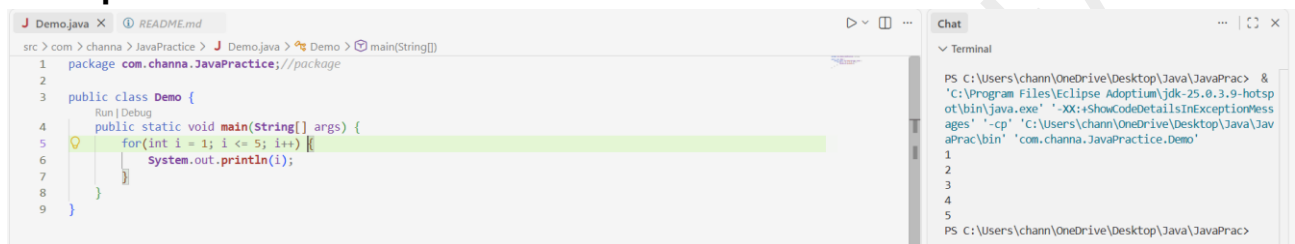
```
1 package com.channa.JavaPractice; //package  
2  
3 public class Demo {  
4     public static void main(String[] args) {  
5         int name = 5;  
6         do {  
7             System.out.println("Channakeshavareddy");  
8             name --;  
9         } while (name <= 0);  
10    }  
11 }
```

↳ **for loop:** Used to **repeatedly execute a block of code** as long as the **specified condition evaluates to true**, primarily when the **number of iterations is known**.

### Syntax:

```
for (initialization; condition; update) {  
    // statements  
}
```

### Example:



The screenshot shows an IDE with a Java file named `Demo.java`. The code is as follows:

```
1 package com.channa.JavaPractice; //package  
2  
3 public class Demo {  
4     public static void main(String[] args) {  
5         for(int i = 1; i <= 5; i++) {  
6             System.out.println(i);  
7         }  
8     }  
9 }
```

The terminal window on the right shows the command to run the program and the output:

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> &  
'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotsp  
ot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMess  
ages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\jav  
aPrac\bin' 'com.channa.JavaPractice.Demo'  
1  
2  
3  
4  
5  
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

## ➤ Transfer Control Statements:



Transfer control statements are used to alter the normal flow of program execution.

### Transfer Control Statements are: -

1. break
2. continue
3. return

➤ **break:** Used to immediately terminates the execution of the nearest enclosing loop and transfers control to the next statement.

#### Example:

```
src > com > channa > JavaPractice > Demo.java > Demo > main(String[])
1 package com.channa.JavaPractice; //package
2
3 public class Demo {
4     public static void main(String[] args) {
5         for(int i = 1; i <= 5; i++) {
6             System.out.println(i);
7             if(i == 3){
8                 break;
9             }
10        }
11    }
12 }
```

Terminal

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
1
2
3
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

➤ **continue:** Used to skip the current iteration of the nearest enclosing loop and continue with the next iteration.

#### Example:

```
src > com > channa > JavaPractice > Demo.java > Demo > main(String[])
1 package com.channa.JavaPractice; //package
2
3 public class Demo {
4     public static void main(String[] args) {
5         for(int i = 1; i <= 5; i++) {
6             if(i == 3){
7                 continue;
8             }
9             System.out.println(i);
10        }
11    }
12 }
```

Terminal

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
1
2
4
5
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

➤ **return:** Used to immediately terminate the execution of the current method and optionally return a value to the caller.

#### Example:

```
src > com > channa > JavaPractice > Demo.java > Demo > main(String[])
1 package com.channa.JavaPractice; //package
2
3 public class Demo {
4     private static int add(){
5         return 10 + 5;
6     }
7     public static void main(String[] args) {
8         int sum = add();
9         System.out.println(sum);
10    }
11 }
```

Terminal

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
15
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

## Scanner Class in Java



The **Scanner class** is a predefined class used to **read user input from the keyboard**.

The Scanner class belongs to the **java.util** package.

↳ **Creating a Scanner class Object:**

**Syntax:** Scanner scan = **new** Scanner(**System.in**);

**Common Input Methods are:**

|                         |                                |
|-------------------------|--------------------------------|
| <b>nextByte()</b>       | <b>Byte</b>                    |
| <b>nextShort()</b>      | <b>Short</b>                   |
| <b>nextInt()</b>        | <b>Int</b>                     |
| <b>nextLong()</b>       | <b>Long</b>                    |
| <b>nextFloat()</b>      | <b>Float</b>                   |
| <b>nextDouble()</b>     | <b>Double</b>                  |
| <b>nextBoolean()</b>    | <b>Boolean</b>                 |
| <b>next()</b>           | <b>A single word (String)</b>  |
| <b>nextLine()</b>       | <b>An entire line (String)</b> |
| <b>next().charAt(0)</b> | <b>A single character</b>      |

# Basic Programs

## Program 1: WAP to Print 1 to N natural Numbers?

```
J Demojava 1 X README.md
src > com > channa > JavaPractice > J Demo.java > Demo > main(String[])

2 import java.util.Scanner;
3 public class Demo {
4     public static void main(String[] args) {
5         Scanner scan = new Scanner(System.in);
6         System.out.print(s: "Enter a positive number: ");
7         int num = scan.nextInt();
8         for(int i = 1; i <= num; i++) {
9             System.out.println(i);
10        }
11    }
12 }
```

Chat

Terminal

Run: Demo

PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'

Enter a positive number: 10

1  
2  
3  
4  
5  
6  
7  
8  
9  
10

PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>

## Program 2: WAP to Print N to 1 natural Numbers?

```
J Demojava 1 X README.md
src > com > channa > JavaPractice > J Demo.java > Demo > main(String[])

2 import java.util.Scanner;
3 public class Demo {
4     public static void main(String[] args) {
5         Scanner scan = new Scanner(System.in);
6         System.out.print(s: "Enter a positive number: ");
7         int num = scan.nextInt();
8         for(int i = num; i >= 1; i--) {
9             System.out.println(i);
10        }
11    }
12 }
```

Chat

Terminal

Run: Demo

PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'

Enter a positive number: 10

10  
9  
8  
7  
6  
5  
4  
3  
2  
1

PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>

## Program 3: WAP to print sum of 1 to N natural Numbers?

```
J Demojava X README.md
src > com > channa > JavaPractice > J Demo.java > Demo > main(String[])

2 import java.util.Scanner;
3 public class Demo {
4     public static void main(String[] args) {
5         Scanner scan = new Scanner(System.in);
6         System.out.print(s: "Enter a positive number: ");
7         int num = scan.nextInt();
8         int sum = 0;
9         for(int i = 1; i <= num; i++){
10             sum += i;
11        }
12        System.out.println("Sum of one to N natural numbers: " + sum);
13        scan.close();
14    }
15 }
```

Chat

Terminal

Run: Demo

PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'

Enter a positive number: 5

Sum of one to N natural numbers: 15

PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>

## Program 4: WAP to print the factorial value of the given number?

```
J Demojava X README.md
src > com > channa > JavaPractice > J Demo.java > Demo > main(String[])

2 import java.util.Scanner;
3 public class Demo {
4     public static void main(String[] args) {
5         Scanner scan = new Scanner(System.in);
6         System.out.print(s: "Enter a positive number: ");
7         int num = scan.nextInt();
8         int factorial = 1;
9         for(int i = 1; i <= num; i++){
10             factorial *= i;
11        }
12        System.out.println("Factorial value of the given number is: " + factorial);
13        scan.close();
14    }
15 }
```

Chat

Terminal

Run: Demo

PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'

Enter a positive number: 5

Factorial value of the given number is: 120

PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>

## Methods in Java



A method is a reusable block of code that performs a specific task.

**Syntax:**     **returnType methodName(parameters) {**  
                    **// code**  
                    **}**

**Example:**

```
public static void greet() {  
    System.out.println("Hello");  
}
```

### Calling the method:

Method calling is the process of invoking a method to execute its defined task.

**Example: -**

```
public class Demo {  
    public static void greet() {  
        System.out.println("Hello");  
    }  
    public static void main(String[] args) {  
        greet();  
    }  
}
```

- **Parameters and method name** together are called the method signature.
- **Access modifier, return type, and method signature** together are called the method declaration.
- **Method declaration and method body** together are called the method definition.

## Return Statement in Methods



The **return** statement is used to terminate a method and return a value to the caller based on return type.

**void** indicates that a **method does not return any value** to the caller.

In Java, methods are commonly classified into 4 types based on parameters and return type.

1. **Method with Parameters & Return Value .**
2. **Method without Parameters & without Return Value.**
3. **Method with Parameters & without Return Value.**
4. **Method without Parameters & Return Value.**

### ➤ Method with Parameters & Return Value:



A method **with parameters and a return value** accepts input through **parameters**, performs a specific task, and **returns a value** to the caller.

**Syntax:**     **returnType methodName(parameter1, parameter2.....) {**  
                    // Method body  
                    **return value;**  
                    **}**

**Example:**

```
1 package com.channa.JavaPractice; //package
2 public class Demo {
3
4     static int add(int a, int b) {
5         return a + b;
6     }
7
8     public static void main(String[] args) {
9         int sum = add(a: 10, b: 20);
10        System.out.println(sum);
11    }
```

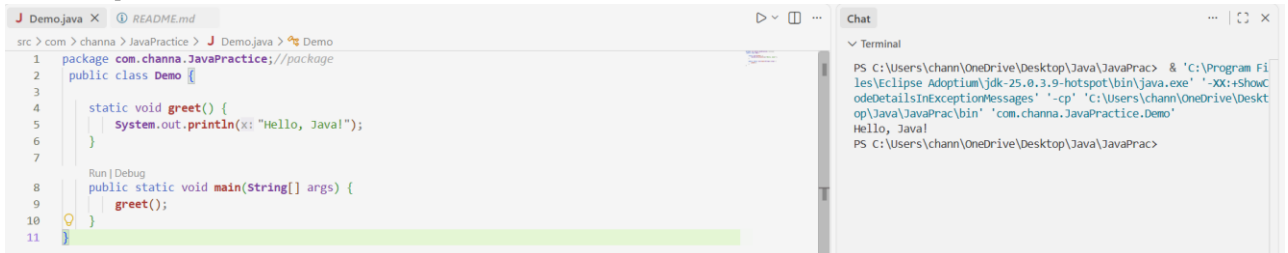
Terminal Output:

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
30
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

## ➤ Method without Parameters & without Return Value:

A method without parameters and without a return value performs a specific task without accepting any input or returning any value to the caller.

### Example:



```
src > com > channa > JavaPractice > J Demo.java > Demo
1 package com.channa.JavaPractice; //package
2 public class Demo {
3
4     static void greet() {
5         System.out.println("Hello, Java!");
6     }
7
8     public static void main(String[] args) {
9         greet();
10    }
11 }
```

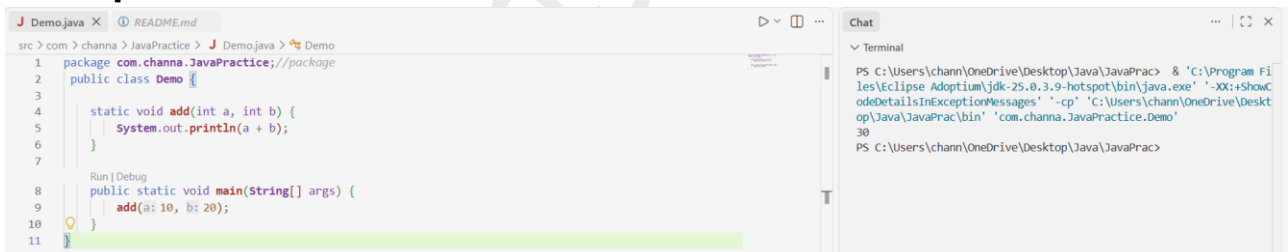
Terminal

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
Hello, Java!
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

## ➤ Method with Parameters & without Return Value:

A method with parameters and without a return value accepts input through parameters but does not return any value to the caller.

### Example:



```
src > com > channa > JavaPractice > J Demo.java > Demo
1 package com.channa.JavaPractice; //package
2 public class Demo {
3
4     static void add(int a, int b) {
5         System.out.println(a + b);
6     }
7
8     public static void main(String[] args) {
9         add(10, 20);
10    }
11 }
```

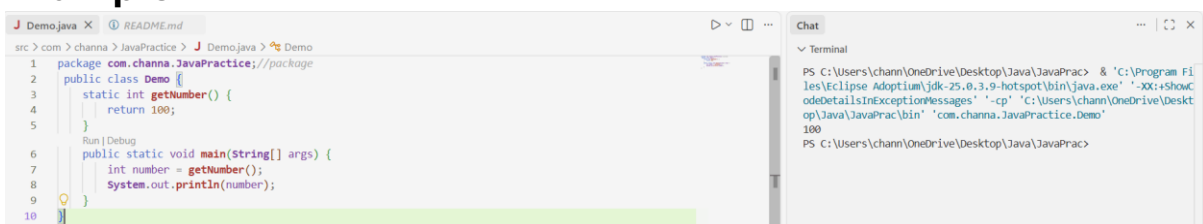
Terminal

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
30
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

## ➤ Method without Parameters & with Return Value:

A method without parameters and with a return value does not accept any input, performs a specific task, and returns a value to the caller.

### Example:



```
src > com > channa > JavaPractice > J Demo.java > Demo
1 package com.channa.JavaPractice; //package
2 public class Demo {
3     static int getNumber() {
4         return 100;
5     }
6
7     public static void main(String[] args) {
8         int number = getNumber();
9         System.out.println(number);
10    }
11 }
```

Terminal

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
100
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

# Method Arguments in Java



Method arguments in Java are classified into two types:

1. Positional arguments
2. Variable-length positional arguments (Varargs)

## ► Positional arguments



**Positional arguments** are arguments passed to a method based on the order or position of the parameters.

**Example:**

```
1 package com.channa.JavaPractice;
2
3 public class Student {
4     static void m1(int num, String name){
5         System.out.println("Age = " + num);
6         System.out.println("Name = " + name);
7     }
8     public static void main(String[] args){
9         m1(num: 22, name: "Channakeshavareddy G N");
10    }
11 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Student'
Age = 22
Name = Channakeshavareddy G N
PS C:\Users\chann\OneDrive\Desktop\Java>
```

## ► Variable-length positional arguments (Varargs...)



**Variable-length positional arguments** allow a method to accept any number of positional arguments.

Java methods use varargs (...) to accept any number of positional arguments.

**Example:**

```
package com.channa.JavaPractice;

public class Student {
    static void m1(int... nums) { // varargs parameter, internally treated as an array
        // it prints the array's default string representation
        System.out.println("varargs should not access like this: " + nums);
        System.out.println("\nvarargs can be access using index: \n" + nums[0]);
        System.out.println(nums[1]);
        System.out.println(nums[2]);
        System.out.println(nums[3]);
        System.out.println(nums[4]);
        System.out.println(nums[5]);
    }
    public static void main(String[] args){
        m1(...nums: 10,20,30,40,50,60);
    }
}
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Student'
varargs should not access like this: [I@7ad041f3
varargs can be access using index:
10
20
30
40
50
60
PS C:\Users\chann\OneDrive\Desktop\Java>
```

# Recursion in Java



**Recursion** is a programming technique in which a **method repeatedly calls itself** until it reaches the **base case**.

The **base case** is the **condition** that **stops the recursion**.

↳ **WAP to print the sum of 1 to N natural numbers using recursion technique?**

```
src > com > channa > JavaPractice > J Demo.java > Demo > sum(int)
1 package com.channa.JavaPractice; //package
2 public class Demo {
3     //sum method is Method with parameters and return value type
4     private static int sum(int num) {
5         if(num == 0){
6             return 0;
7         } else {
8             return num + sum(num - 1); // sum(num - 1); recursive call.
9         }
10    }
11
12    Run | Debug
13    public static void main(String[] args) {
14        int sum = sum(10); //method call by passing 10 as an argument
15        System.out.println(sum);
16    }
17 }
```

Chat

Terminal

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
55
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

↳ **WAP to print the Factorial numbers using recursion technique?**

```
src > com > channa > JavaPractice > J Demo.java > Demo > main(String[])
1 package com.channa.JavaPractice; //package
2 public class Demo {
3     //factorial method is Method with parameters and return value type
4     private static int factorial(int num) {
5         if(num == 1){
6             return 1;
7         } else {
8             return num * factorial(num - 1); // sum(num - 1); recursive call.
9         }
10    }
11
12    Run | Debug
13    public static void main(String[] args) {
14        int factorial = factorial(10); //method call by passing 10 as an argument
15        System.out.println("Factorial value of n number is: " + factorial);
16    }
17 }
```

Chat

Terminal

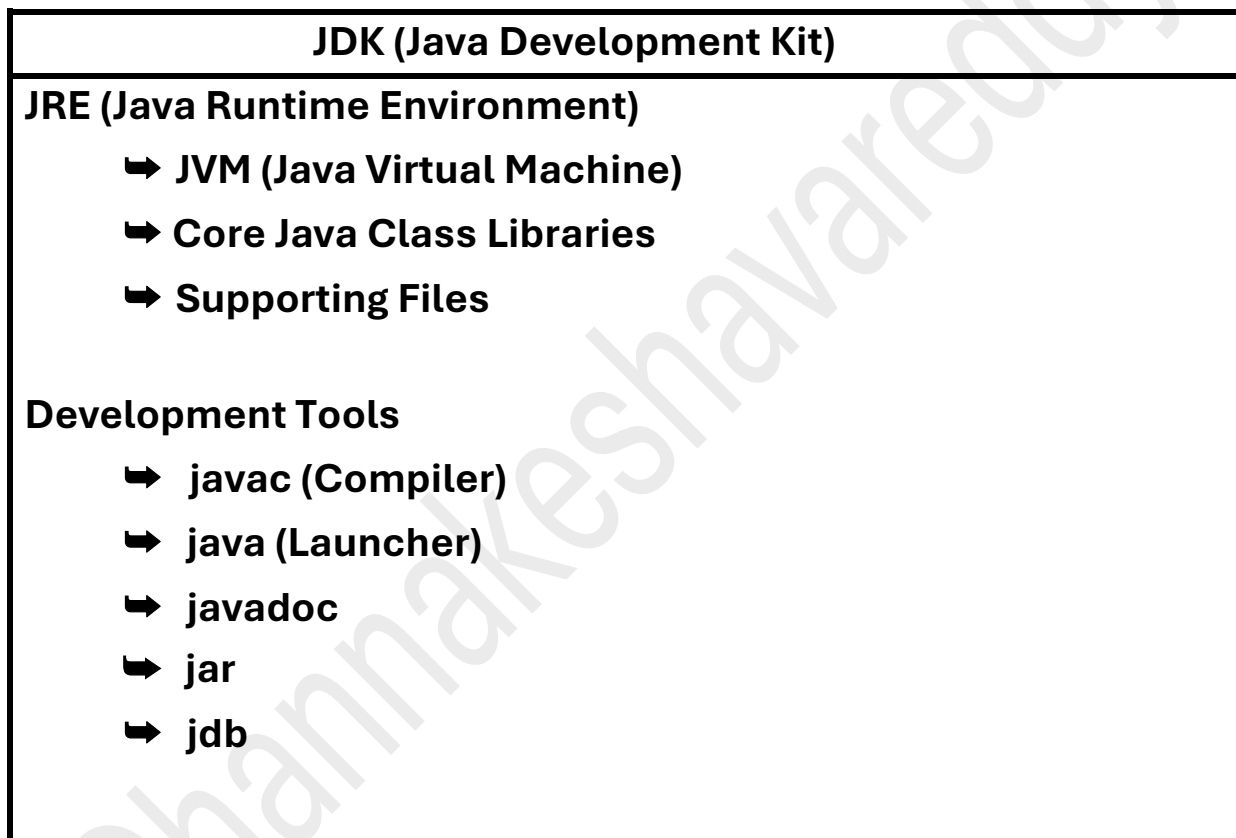
```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
Factorial value of n number is: 120
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

## JDK, JRE & JVM in Java



### JAVA PROGRAM EXECUTION ARCHITECTURE

Java Program (.java)



## **1. Class Loader Subsystem**

- ↳ **Bootstrap ClassLoader**
- ↳ **Extension ClassLoader**
- ↳ **Application ClassLoader**
- ↳ **Parent Delegation Model**

## **2. Runtime Memory Areas**

- ↳ **Heap**
- ↳ **Stack**
- ↳ **Method Area**
- ↳ **PC Register**
- ↳ **Native Method Stack**

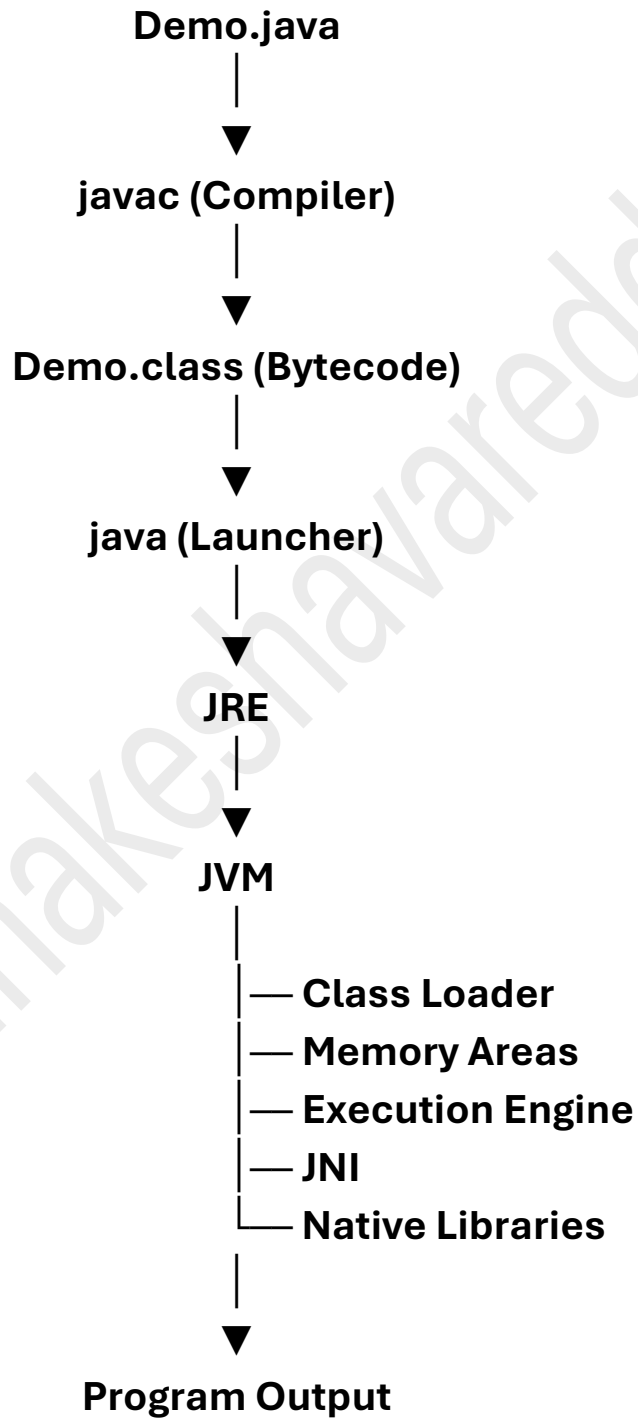
## **3. Execution Engine**

- ↳ **Interpreter**
- ↳ **JIT Compiler**
- ↳ **Garbage Collector**

## **4. Java Native Interface (JNI)**

## **5. Native Method Libraries**

## Complete Execution Flow:



## ↳ **JDK: Java Development Kit.**



**JDK** is known as **Java Development Kit**.

It is a **software development kit** used to develop and run Java programs.

### **JDK consists of:**

1. **JRE** (Java Runtime Environment)

2. Development Tools:

↳ **javac** (Compiler)

↳ **java** (Launcher / Command-line tool)

↳ **javadoc** (Documentation Generator)

↳ **jar** (Java Archive Tool)

↳ **jdb** (Java Debugger)

↳ Other development tools

**The relationship is that the JDK includes the JRE along with development tools.**

### **1. What is javac in JDK?**

**javac** is a Development tool inside JDK.

**javac** is a java compiler. It converts java **source code (.java)** into **bytecode (.class)**.

Without **javac**, Java source code cannot be compiled into bytecode.

### **2. What is java Launcher / Command line tool in JDK?**

The java Launcher is a command-line tool in the JDK used to invoke (call) the JRE.

The JRE, starts the JVM to load and execute Java bytecode (.class).

**Note:** The JVM can be started by the java Launcher or by any other tool that invokes the JRE.

### 3.What is javadoc in JDK?

**javadoc** is documentation generation tool.

It is used to create documentation from java source code.

It reads comments in Java source file and generates HTML documentation.

**javadoc** works only with documentation comments.

#### Example: /\*\*

```
This is class demonstrates
```

```
*/
```

```
public class Add {
```

```
/**
```

```
This is method add two numbers
```

```
*/
```

```
public static int add(int num1, int num2){
```

```
    return num1 + num2;
```

```
}
```

```
}
```

### 4.What is .jar in JDK?

**jar** is known as Java Archive.

It is a development tool used to package multiple **.class** files and resources into a single **.jar** file.

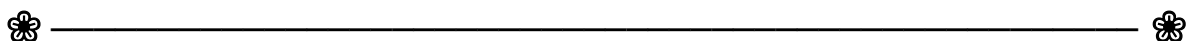
### 5. What is jdb in JDK?

**jdb** is a Java debugger tool provided by the JDK that is used to find and fix errors in Java programs.

#### Features:

- ↳ Set breakpoints.
- ↳ Inspect variable values.
- ↳ Control program execution

### ↳ JRE: Java Runtime Environment.



**JRE** is a software environment that provides the runtime environment to execute Java bytecode.

#### **JRE consists of:**

- ↳ JVM (Java Virtual Machine)
- ↳ Core Java Class Libraries
- ↳ Supporting files required to run Java programs.

↳ **JVM:** Java Virtual Machine.



**JVM is an abstract machine that provides the runtime environment to execute Java bytecode.**

**JVM consists of:**

- ↳ ClassLoader Subsystem
- ↳ Memory Areas
- ↳ Execution Engine
- ↳ Java Native Interface (JNI)
- ↳ Native Method Libraries

## ↳ **1. ClassLoader Subsystem in JVM?**

The **ClassLoader Subsystem** is a module of the **JVM** that loads Java bytecode (.class files) into JVM memory using different class loaders.

**The main ClassLoaders are:**

- ↳ **Bootstrap ClassLoader**
- ↳ **Platform ClassLoader**
- ↳ **Application ClassLoader**

### **1. What is the Bootstrap ClassLoader?**

**Bootstrap ClassLoader** is the parent class loader that loads core Java classes into the **JVM**.

**Examples:**

- java.lang.String
- java.lang.Object
- java.lang.System

### **2. What is the Platform ClassLoader?**

**Platform ClassLoader** is a **JVM class loader** that loads platform classes and Java platform modules into the **JVM**.

**Examples:**

- java.sql.DataSource
- java.sql.Driver
- java.xml.parsers.DocumentBuilder

### 3. What is the Application ClassLoader?

**Application ClassLoader** is a JVM class loader that loads user-defined classes into the JVM.

#### **Examples:**

- Main
- Student
- Hello

#### **↳ Parent Delegation Model:**

The **Parent Delegation Model** is a mechanism in which a **ClassLoader** first asks its **parent ClassLoader** to load a class before trying to load it itself.

The purpose of the **Parent Delegation Model** is to **ensure security, prevent duplicate class loading**, and maintain the class loading hierarchy in the JVM.

#### **Flow of the Parent Delegation Model:**

**Step 1:** Application ClassLoader receives the class loading request.

**Step 2:** It delegates the request to the Platform ClassLoader.

**Step 3:** The Platform ClassLoader delegates the request to the Bootstrap ClassLoader.

**Step 4:** The Bootstrap ClassLoader tries to load the class.

**Step 5:** If found → Class loaded.

**Step 6:** If not found, control returns to the Platform ClassLoader.

**Step 7:** The Platform ClassLoader tries to load the class.

**Step 8:** If found → Class loaded.

**Step 9:** If not found, control returns to the Application ClassLoader.

**Step 10:** The Application ClassLoader tries to load the class.

**Step 11:** If no ClassLoader can load the class, the JVM throws **ClassNotFoundException**.

## ↳ 2. Memory Areas in JVM:

**JVM memory areas are the runtime memory regions** where the JVM stores **objects, class information, method data, and execution information** during program execution.

The purpose of **JVM memory areas is to support the execution** of Java bytecode.

**JVM has five main memory areas:**

- **Heap**
- **Stack**
- **Method Area**
- **PC Register**
- **Native Method Stack**

### 1). What is Heap memory in JVM?

**Heap memory is the runtime memory area in the JVM** where **objects, arrays, and instance variables** are dynamically allocated.

### 2). What is Stack in JVM?

**Stack memory is the runtime memory area in JVM** that stores **method calls, Local variables, and references to objects in heap**, following **LIFO (Last in First Out) Order**.

### 3). What is the Method Area in the JVM?

**Method Area is the runtime memory area in the JVM** that stores **class-level information, including method bytecode, static variables, constants, and metadata**.

## ↳ What is metadata?

**Metadata is runtime information about classes, methods, and fields** used by the **JVM** during program execution.

### 4). What is PC – Register in JVM memory?

**PC – Register stores the address of the current bytecode instruction** for each thread in **JVM**.

## 5. What is the Native Method Stack in the JVM?

**Native Method Stack** is a **JVM memory area** that stores the **execution state, local variables, parameters, and call information of native methods** invoked by a Java program.

## ➤ 3. JVM Execution Engine:

**Execution Engine** is a **component of the JVM** that executes Java bytecode by converting it into machine code.

**Execution Engine consists of:**

- ↳ Interpreter
- ↳ JIT (Just-In-Time) Compiler
- ↳ Garbage Collector (GC)

## 1. What is the Interpreter in the JVM Execution Engine?

Interpreter is a component of the JVM Execution Engine that reads and executes **Java bytecode instruction by instruction (line by line)**.

**The Interpreter is slower than the JIT Compiler** because it executes bytecode one instruction at a time.

## 2. What is the JIT (Just-In-Time) Compiler in the JVM Execution Engine?

JIT Compiler is a component of the JVM Execution Engine; **it converts Java bytecode into native machine code at runtime for faster execution.**

JIT improves performance by executing frequently used code as native machine code.

## 3. What is the Garbage Collector (GC) in the JVM Execution Engine?

Garbage Collector (GC) is a component of the JVM Execution Engine that automatically removes unused or unreferenced objects from the heap memory to free memory.

#### ↳➤ 4. Java Native Interface (JNI) in the JVM:

JNI is a bridge that allows Java programs to call native code written in languages such as C and C++.

It is mainly used to integrate Java with native libraries and access low-level system features.

#### ↳➤ 5. Native Method Libraries in the JVM:

Native Method Libraries are collections of pre-compiled native code written in C or C++ that are accessed by Java programs through JNI.

They provide native functionality that Java uses through JNI to access low-level system features.

**Final Summary:** JVM executes Java programs using the **ClassLoader Subsystem, Memory Areas, Execution Engine, and Native Support (JNI & Native Method Libraries).**

#### ↳➤ JRE Core Java Class Libraries:



Core Java Class Libraries are a collection of pre-defined packages and classes provided by the JRE that offer standard functionality such as

- data handling,
- input/output (I/O),
- collections,
- networking, and more.

**These libraries are used by Java programs at runtime.**

#### ↳➤ JRE Supporting files:



Supporting files are non-Java files in JRE such as configuration, security and Native libraries that assist JVM and class libraries in executing java programs correctly.

 **JAVA FUNDAMENTALS COMPLETED** 

**Status** : ✓ Completed

**Foundation** : ✓ Strong

**Next Module** : Object-Oriented Programming (OOP)

**“Excellence in software starts with mastering the fundamentals”**

---